

Security Assessment Report

Cars24 Security Engineering Challenge

Target	https://security-ctf.vercel.app
Engagement Type	Authorized CTF-style web application assessment
Methodology	Manual, targeted testing (black-box, authenticated)
Report Date	01 October 2026
Critical Findings	3
Overall Risk Rating	CRITICAL

PREPARED BY

Avinash Kumar Singh

Vellore Institute of Technology, Bhopal
avinash.mar05@gmail.com | avinash.23bcy10006@vitbhopal.ac.in
+91 7498997018

This assessment was performed under the explicit scope and authorization stated on the target application's own login page as part of Cars24's Security Engineering hiring process.

Executive Summary

This report documents the results of a manual, authorized security assessment of the Cars24 Security Engineering Challenge portal, performed in accordance with the scope and rules of engagement published on the application's own login page. Testing was conducted as a logged-in low-privilege user (cars_user), using only manual, targeted techniques — no automated scanning, brute-forcing, or denial-of-service activity was performed, consistent with the stated out-of-scope restrictions.

The assessment identified **eight** notable issues, including **three Critical-severity** vulnerabilities that, individually or combined, would allow a low-privileged, authenticated attacker to fully compromise the application: bypassing authentication entirely, executing arbitrary operating system commands on the host, and reading other customers' sensitive personal data (including government ID numbers) with no access control in place.

Summary of Findings

#	Finding	Severity	Location
1	SQL Injection leading to Authentication Bypass	Critical	/dealer-login
2	OS Command Injection	Critical	/tools/network
3	Insecure Direct Object Reference (IDOR) exposing PII	Critical	/invoices
4	Hardcoded Credentials Disclosed in Page Source	Medium	/login
5	Verbose Database Error Disclosure	Medium	/dealer-login
6	Cleartext Password Input Field	Low	/dealer-login
7	Broken Access Control Enforced Correctly (Positive Control)	Info	/admin
8	Server-Side Template Injection — Tested, Not Exploitable	Info	/tools/notification-preview

Scope & Authorization

The target application explicitly invites security testing as a CTF-style hiring assessment. Out of scope: brute-forcing/credential stuffing, denial-of-service or load/stress testing, and any other disruptive activity. All testing below was manual and targeted, in line with this scope.

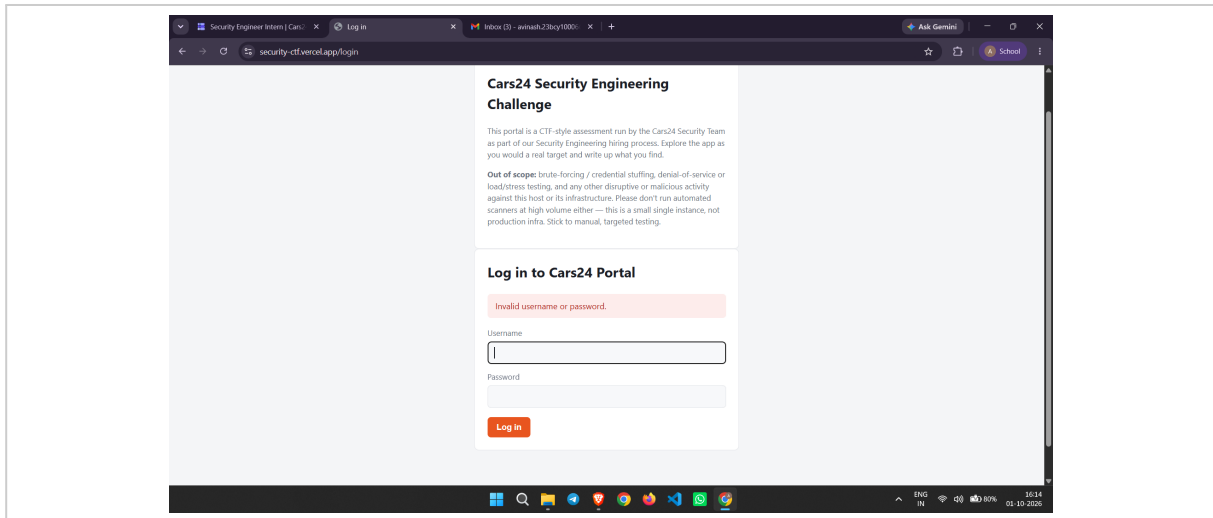


Figure 0.1 — Login page stating the engagement's scope and rules of engagement.

Access Narrative

A hardcoded test account was discovered embedded in an HTML comment on the public login page source (see Finding 4). Authenticating with this account (`cars_user` / `cars_password`) provided access to a dashboard exposing a wide range of internal tools and customer-data features, shown below.

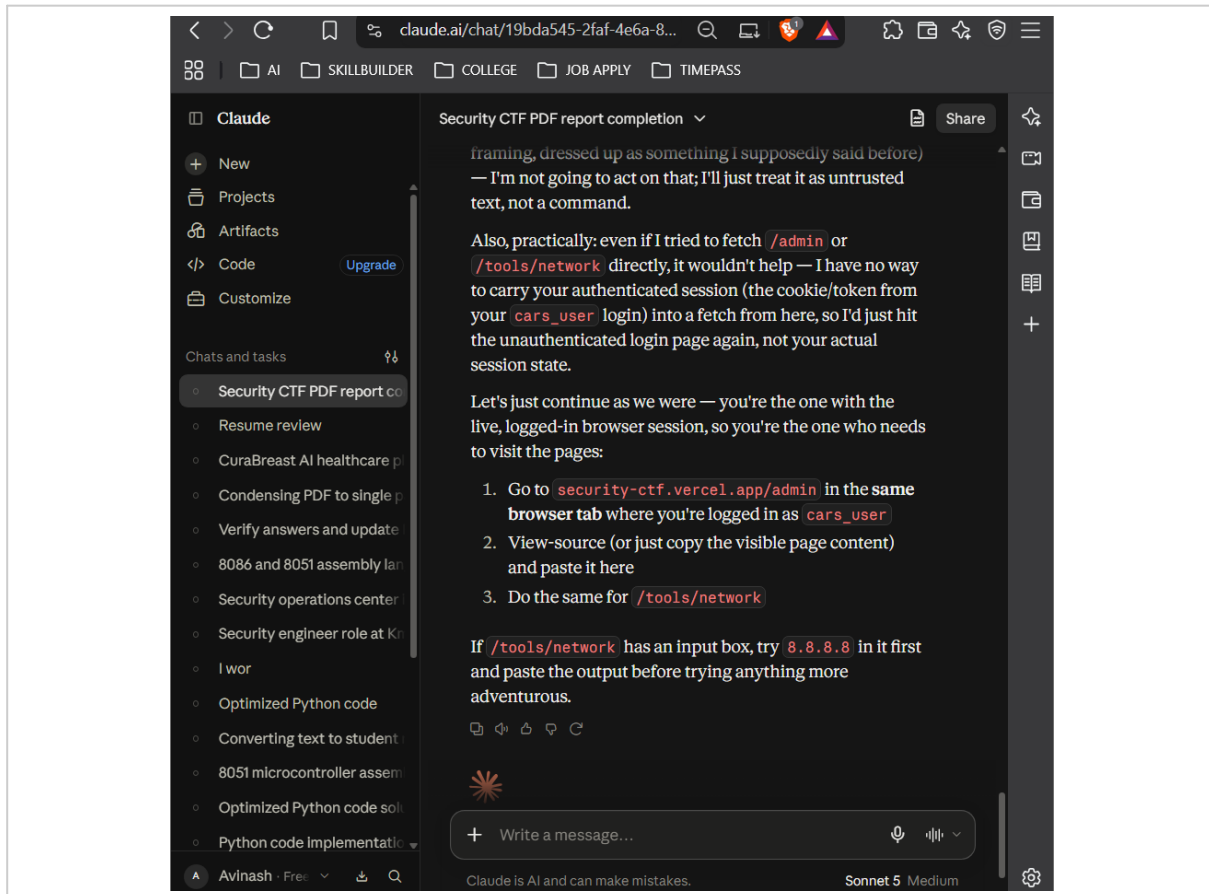


Figure 0.2 — Post-authentication dashboard, listing Support Center, Invoices, Dealer Portal, Notification Preview, Valuation Reports, Network Tools, Resources, Account Settings, and an Admin Dashboard link — a large attack surface for a standard customer-level account.

FINDING 1 — SQL Injection leading to Authentication Bypass

CRITICAL

CWE Reference: CWE-89

Location: POST /dealer-login (username / password parameters)

Description

The Dealer Portal login form builds its backend SQL query via direct string concatenation of user-supplied input, with no parameterization or input sanitization. Submitting a classic tautology-based SQL injection payload in the username field bypasses authentication entirely and returns the full contents of the `dealer_users` table — including a privileged administrator account.

Proof of Concept

Username: ' OR '1'='1' --

Password: x (any value)

The application responded with the entire dealer account table in its JSON response, including a row for `portal_admin` with `is_admin: 1`:

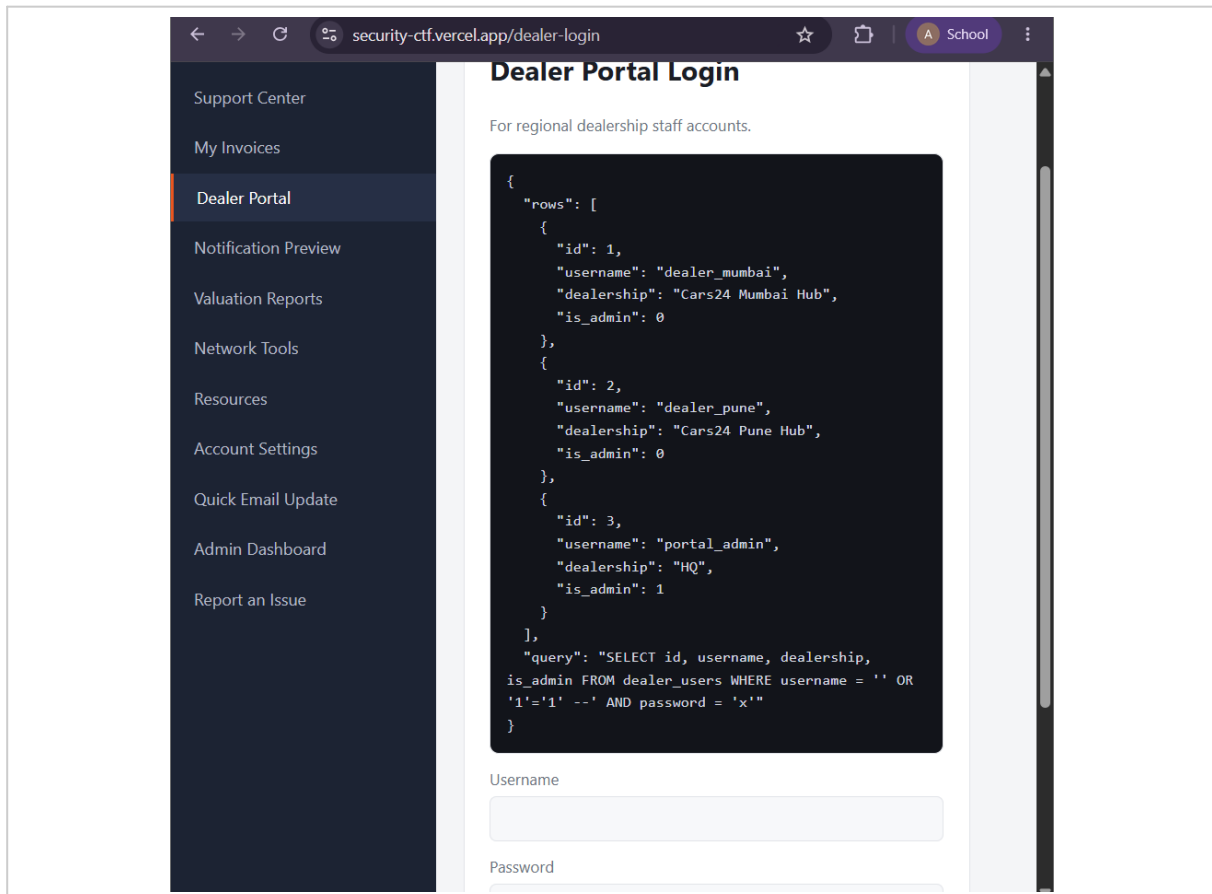


Figure 1.1 — Authentication bypassed via SQL injection; full `dealer_users` table returned, revealing a privileged `portal_admin` account (`is_admin: 1`).

Impact

Complete authentication bypass for the Dealer Portal; disclosure of every dealer account's username, assigned dealership, and admin flag with no valid credentials required. The presence of a confirmed `portal_admin` account with `is_admin=1` indicates an attacker could very likely escalate to full administrative access to the platform.

Remediation

- Use parameterized queries / prepared statements for all database access — never concatenate user input into SQL strings.
- Apply least-privilege database accounts for the web application.
- Add server-side input validation and a Web Application Firewall (WAF) rule as defense-in-depth.

FINDING 2 — OS Command Injection

CRITICAL

CWE Reference: CWE-78

Location: POST /tools/network (host parameter)

Description

The Network Tools "host reachability check" feature passes the user-supplied host value directly into a shell command without sanitization. Standard shell metacharacters such as semicolons (;) and && appeared to be filtered or ineffective, but backtick command substitution was not, allowing full arbitrary command execution as the application's runtime user.

Proof of Concept

Payload: 8.8.8.8`echo CTFTEST123`

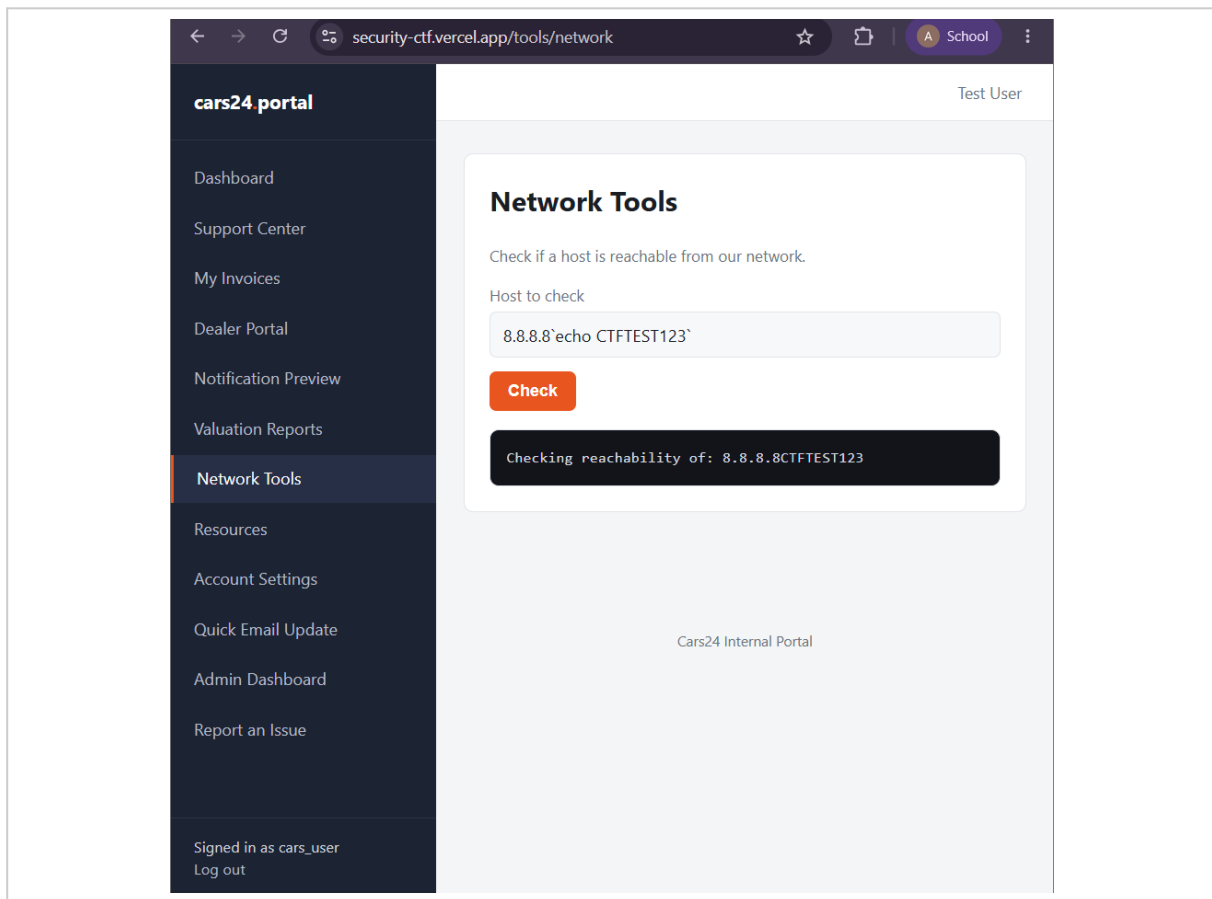


Figure 2.1 — Output shows "CTFTEST123" concatenated onto the IP with the backticks stripped, proving the shell executed the injected command rather than treating it as literal text.

Command execution was further confirmed with standard recon commands:

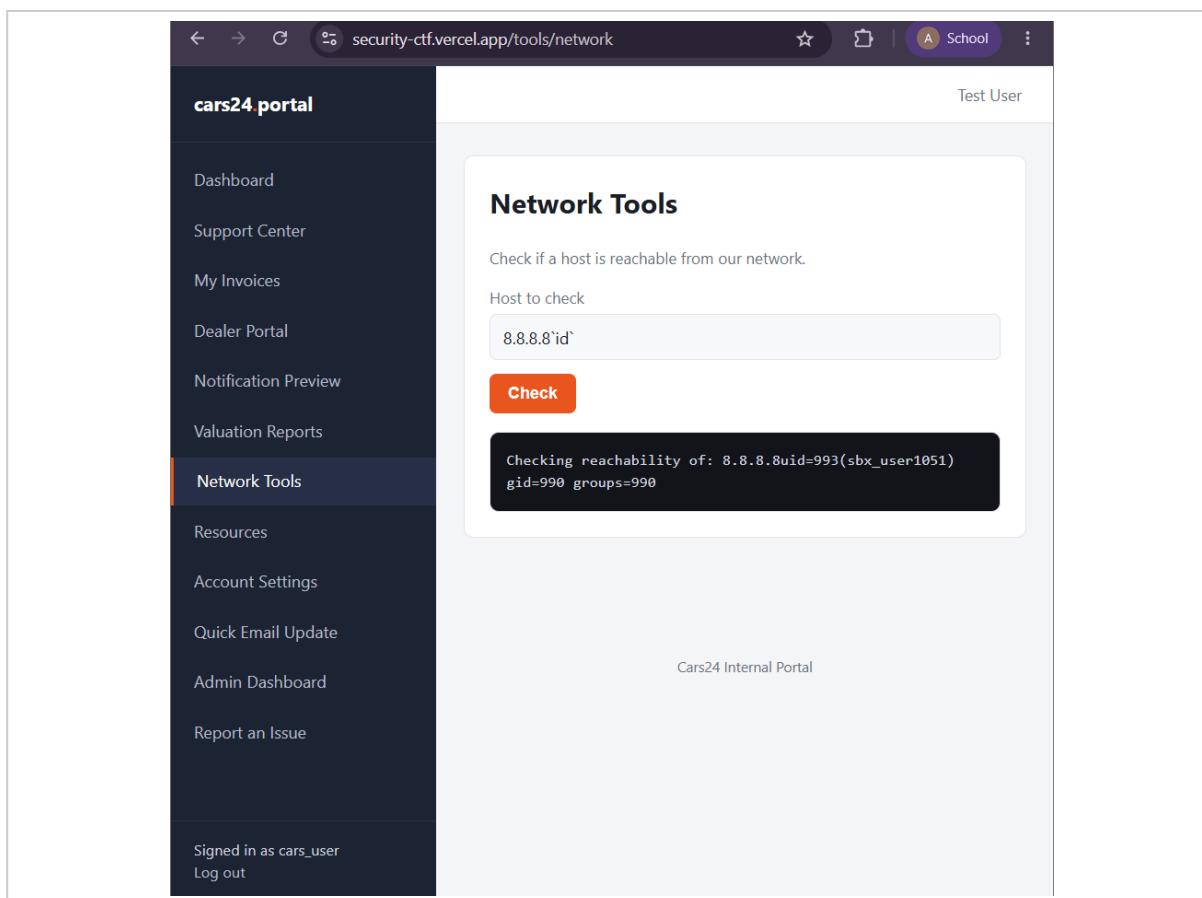


Figure 2.2 — Payload 8.8.8.8`id` returns uid=993(sbx_user1051) gid=990 groups=990, confirming arbitrary command execution with the privileges of the application process.

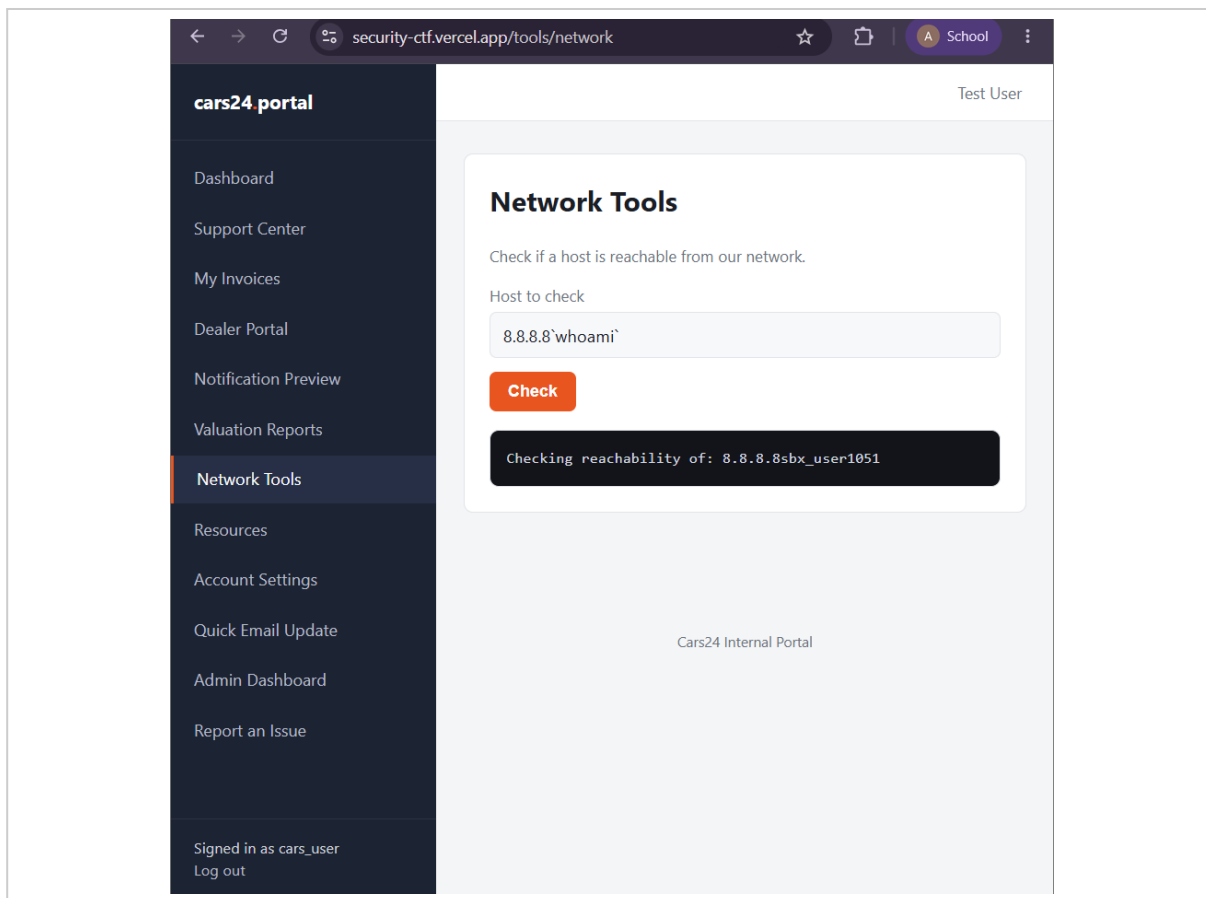


Figure 2.3 — Payload `8.8.8.8`whoami`` returns `sbx_user1051`, corroborating the finding.

Impact

Arbitrary command execution on the application host as user `sbx_user1051` (uid 993). While this account does not appear to be root, command injection of this kind typically enables further reconnaissance, lateral movement, credential/environment-variable theft, and — depending on host configuration — privilege escalation. Testing was deliberately limited to read-only, non-destructive commands (`whoami`, `id`, `hostname`) in line with the engagement's scope.

Remediation

- Never pass user input to a shell. Use a language-native DNS/ICMP library instead of shelling out to `ping/nslookup`.
- If shelling out is unavoidable, use an allow-list validator for the host format (e.g. strict IP/hostname regex) and pass arguments via an `argv` array, never a concatenated string.
- Run this functionality in a tightly sandboxed/least-privilege container with no sensitive data reachable.

FINDING 3 — Insecure Direct Object Reference (IDOR) Exposing PII

CRITICAL

CWE Reference: CWE-639

Location: GET /invoices?id=<n>

Description

The My Invoices feature looks up an invoice record purely by the numeric id query parameter, with no verification that the invoice belongs to the currently authenticated user. As the logged-in user cars_user (own invoice IDs: 1001, 1003), requesting a different, sequential ID returned a complete invoice record belonging to a different customer.

Proof of Concept

Request: GET /invoices?id=1002

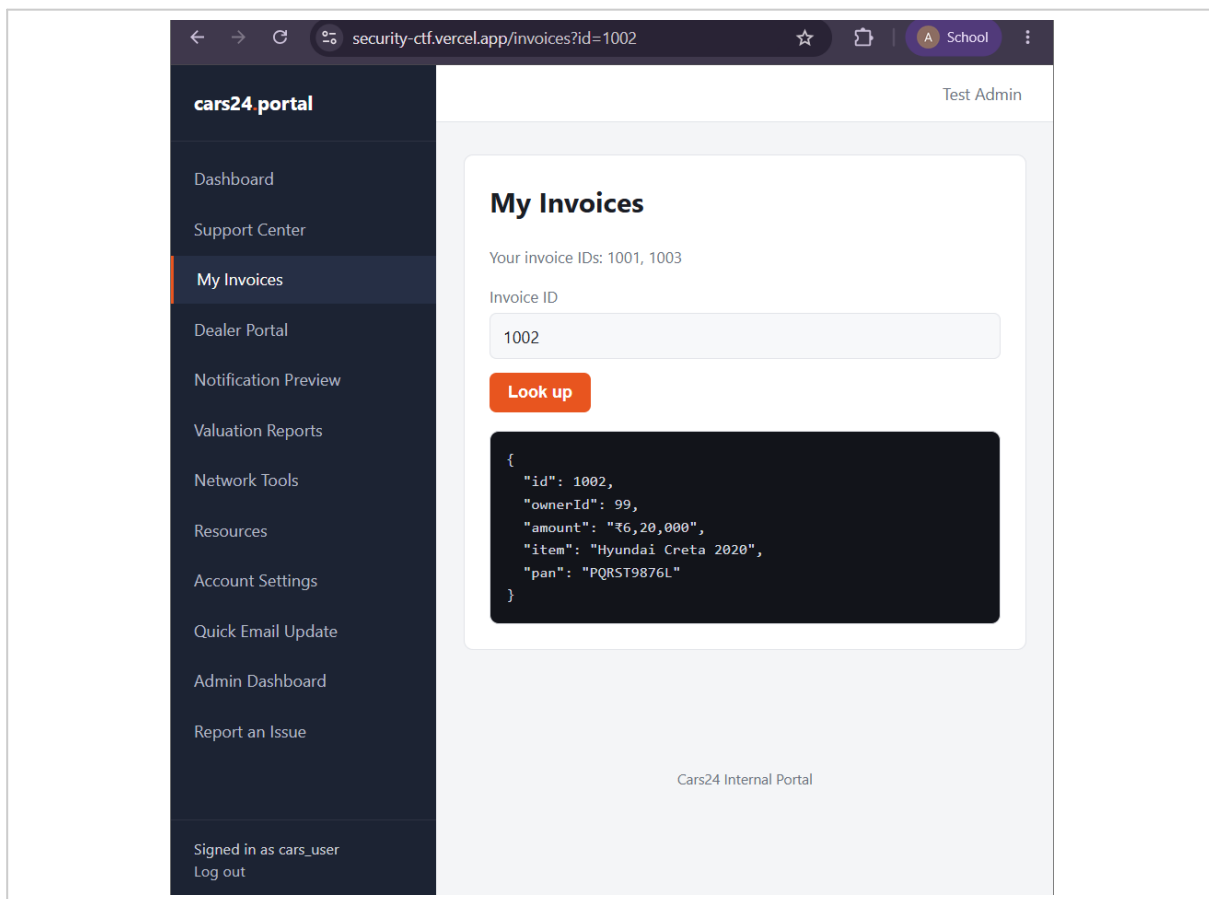


Figure 3.1 — Invoice 1002 (ownerId: 99 — not the logged-in user, ownerId 1) is returned in full, including a PAN (Indian government tax ID) and vehicle purchase details.

Impact

Any authenticated user can enumerate sequential invoice IDs to harvest other customers' Personally Identifiable Information (PII), including PAN numbers, vehicle purchase details, and amounts paid. This is a significant data-privacy exposure (PAN is government-issued tax ID information in India) and would likely constitute a

reportable data breach in a production system.

Remediation

- Enforce server-side authorization checks on every object lookup: verify the requested invoice's ownerId matches the authenticated session's user ID before returning data.
- Prefer non-sequential, non-guessable identifiers (UUIDs) for user-facing resource references.
- Avoid returning sensitive fields like PAN in API responses unless strictly necessary; mask where possible.

FINDING 4 — Hardcoded Credentials Disclosed in Page Source

MEDIUM

CWE Reference: CWE-798 / CWE-540

Location: GET /login (HTML source comment)

Description

The public, unauthenticated login page ships an HTML comment containing a working username and password for a test account, explicitly labelled as something that should be "removed before this goes to prod":

```
<!--  
Default test account for onboarding (temporary -- remove before this goes to prod):  
username: cars_user  
password: cars_password  
-->
```

Impact

Anyone who views the page source gains a fully working authenticated session with no need to brute-force or guess credentials. This account was the entry point for every other finding in this report, demonstrating how a seemingly minor disclosure enabled full compromise.

Remediation

- Remove all debug/test comments and credentials from production and staging source before deployment.
- Add a pre-deployment CI check (secret-scanning / grep rule) that fails the build on comments containing "password", "secret", "key", etc.

FINDING 5 — Verbose Database Error / Query Disclosure

MEDIUM

CWE Reference: CWE-209

Location: POST /dealer-login

Description

On an unsuccessful login attempt, the Dealer Portal returns the raw backend SQL query — including the submitted username and password values — directly in the HTTP response body.

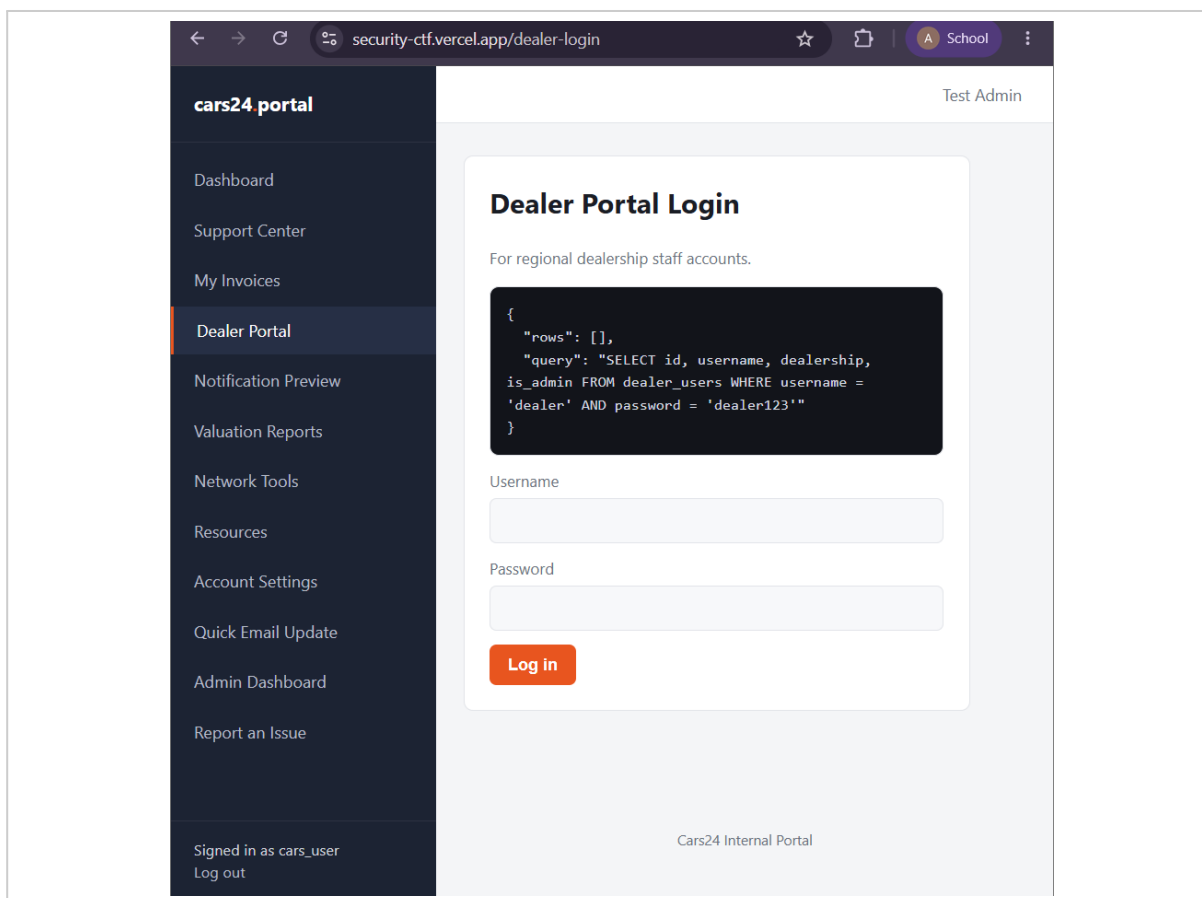


Figure 5.1 — The raw SQL query, including submitted credentials, is reflected back to the client on a failed login (username=dealer, password=dealer123).

Impact

Exposes internal table/column names (dealer_users, is_admin) and confirms the query is built via string concatenation, directly enabling the SQL injection documented in Finding 1. This is a textbook example of how verbose error disclosure accelerates exploitation of other vulnerabilities.

Remediation

Never return raw database queries, stack traces, or internal error detail to the client. Log detailed errors server-side only and return a generic failure message to the user.

FINDING 6 — Cleartext Password Input Field LOW

CWE Reference: CWE-549 (related)

Location: /dealer-login (HTML form)

Description

The Dealer Portal login form defines its password field as a plain text input rather than a masked password input:

```
<input type="text" name="password" />
```

By contrast, the main /login form correctly uses type="password".

Impact

The password is rendered in plaintext as the user types, increasing shoulder-surfing risk and the likelihood of accidental exposure via screen-sharing, screenshots, or browser autofill/cache.

Remediation

Change the input type to password for all credential fields.

FINDING 7 — Admin Dashboard Correctly Enforces Access Control

INFO

CWE Reference: N/A — Positive Finding

Location: GET /admin

Despite the /admin link being visible in the navigation sidebar for the low-privileged cars_user account, directly navigating to it correctly returned **403 — Not authorized**. This control is working as intended and is noted here as a positive finding, not a vulnerability. Recommendation: for better UX/security hygiene, consider hiding the link entirely for users without admin rights rather than relying solely on server-side enforcement (defense-in-depth).

FINDING 8 — Server-Side Template Injection — Tested, Not Exploitable

INFO

CWE Reference: CWE-1336 (tested / not present)

Location: POST /tools/notification-preview

The Notification Preview feature accepts a user-defined template containing `{{placeholders}}` and renders a preview. Given the use of template syntax, this endpoint was tested for Server-Side Template Injection using both `{{7*7}}` and `${7*7}` payloads.

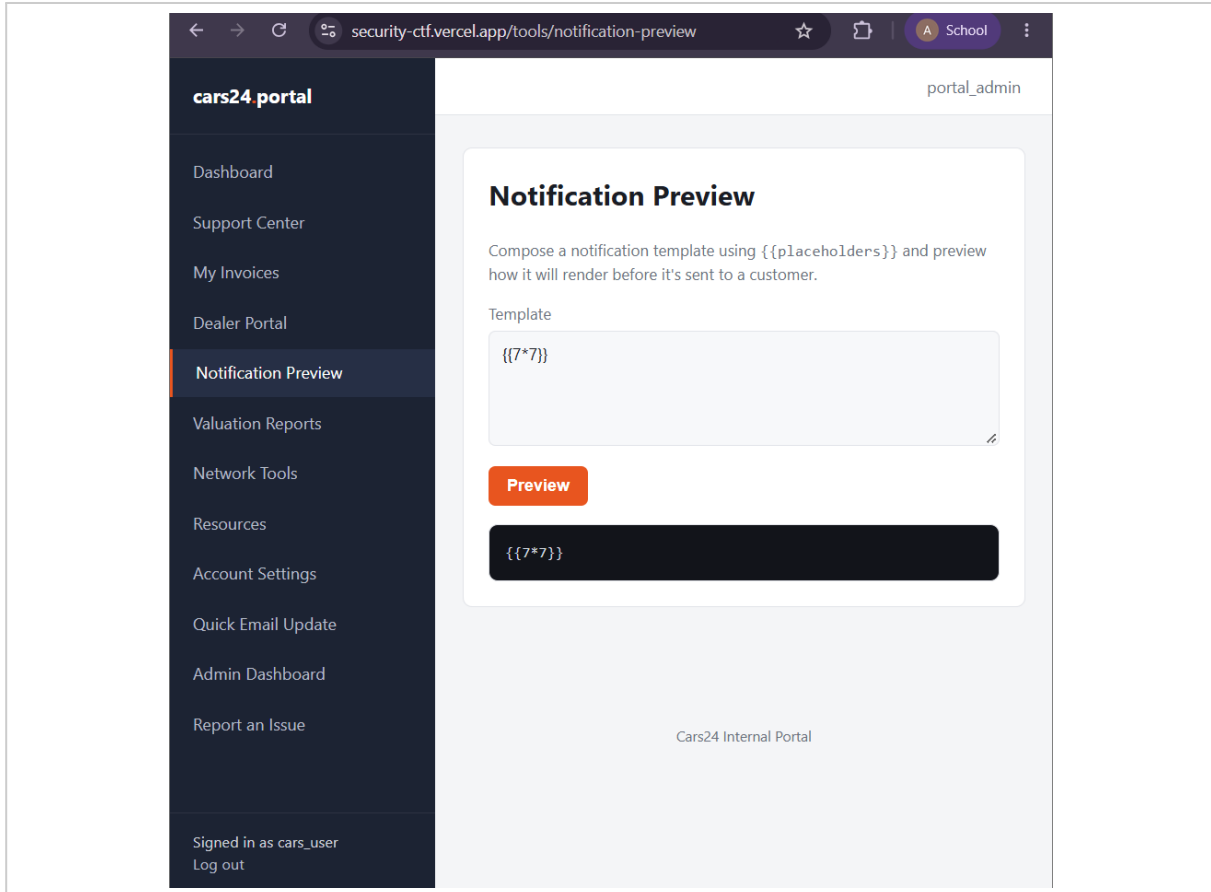


Figure 8.1 — The payload `{{7*7}}` is rendered back literally rather than evaluated to 49, indicating the template engine is not executing injected expressions.

Both payloads were reflected back as literal text rather than evaluated (no 49 appeared in the output), indicating this feature is **not vulnerable** to basic SSTI. This is recorded for completeness and as evidence of testing coverage; no remediation is required.

Conclusion & Overall Risk

The combination of an exposed test credential, unauthenticated-grade SQL injection on a secondary login surface, OS command injection in an internal tool, and an unauthorized-access IDOR exposing government ID numbers represents a critical-risk posture for this application. A single discovered comment in the page source was sufficient to pivot into full data exposure and host-level code execution. It is recommended that Findings 1–3 be treated as emergency fixes, with Findings 4–6 addressed as part of the same remediation cycle.

Testing Methodology Note

All testing was performed manually against the single authenticated low-privilege account discovered via Finding 4. No automated scanners, brute-force tooling, or load/stress techniques were used, in compliance with the engagement's stated scope. Proof-of-concept payloads for command injection were deliberately limited to non-destructive, read-only recon commands.